

实验十三 降维方法

（一）目的和要求

- （1）理解降维的概念，明确降维在数据处理和机器学习中的作用与意义。
- （2）掌握降维的应用，了解降维在实际问题中的具体应用场景。
- （3）学习常用降维方法，熟悉并掌握几种常用的降维技术及其实现方法。

（二）实验内容及步骤

（1）知识准备

1. 降维的概念

降维是指通过数学方法将高维数据映射到低维空间的过程。在数据处理和机器学习中，高维数据往往包含大量冗余信息，这些信息不仅增加了计算的复杂度，还可能影响模型的性能。降维技术通过保留数据的主要特征，减少数据的维度，从而提高计算效率和模型性能。

2. 降维的应用

- （1）数据可视化：将高维数据降维到二维或三维，以便于可视化展示。
- （2）去噪和特征提取：通过降维去除数据中的噪声，提取主要特征。
- （3）提高计算效率：减少数据的维度，降低计算复杂度，提高算法的运行速度。
- （4）避免过拟合：在高维数据中，模型容易过拟合，降维有助于减少过拟合的风险。

3. 常用的降维方法

- （1）主成分分析（PCA）：通过线性变换将数据投影到主成分上，保留数据的主要方差。
- （2）线性判别分析（LDA）：在降维的同时考虑类别信息，使同类数据尽可能接近，异类数据尽可能远离。
- （3）t-分布随机邻域嵌入（t-SNE）：一种非线性降维方法，适用于高维数据的可视化。
- （4）自编码器（Autoencoder）：通过神经网络学习数据的低维表示。

（2）实验步骤

1. PCA 降维及应用

理解 PCA 降维的基本原理，并应用于数据可视化。生成或加载一个高维数据集。使用 PCA 进行降维。可视化降维后的数据。

示例代码：

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.decomposition import PCA
4 from sklearn.datasets import make_blobs
5
6 # 生成模拟数据
7 X, y = make_blobs(n_samples=100, n_features=10, centers=3, random_state=42)
8
9 # PCA降维
10 pca = PCA(n_components=2)
11 X_reduced = pca.fit_transform(X)
12
13 # 可视化
14 plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=y, cmap='viridis')
15 plt.xlabel('Principal Component 1')
16 plt.ylabel('Principal Component 2')
17 plt.title('PCA of High-Dimensional Data')
18 plt.colorbar(label='Cluster')
19 plt.show()
```

示例结果如下：

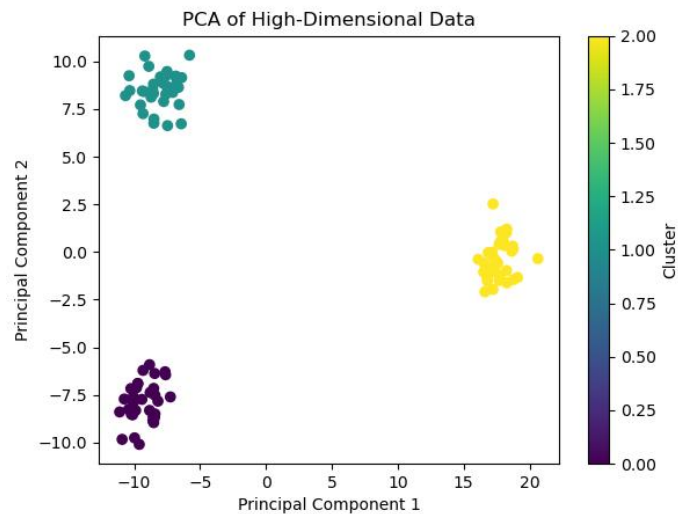


图 1 PCA 降维

图 1 展示了高维数据通过 PCA 降维至二维空间后的可视化结果，并结合聚类分析揭示了数据的内在结构。其中，主成分 1 和主成分 2 是原始高维数据中方差最大的两个方向，共同捕获了数据的主要变化模式。数据点在主成分 1 和主成分 2 构成的坐标系中自然形成三个颜色区分的簇，表明数据存在显著分组特征：黄色簇在主成分 1 高值区域独立分布，而青绿色与紫色簇虽在主成分 1 上重叠，但通过主成分 2 实现清晰分离，说明 PCA 有效提取了区分不同群体的核心特征，同时聚类结果反映了数据潜在的分类模式或结构差异，为进一步分析特征重要性或类别归属提供了直观依据。

2. LDA 降维及分类性能分析

理解 LDA 降维的基本原理，并分析降维对分类性能的影响。加载一个多分类数据集（如 Iris 数据集）。使用 LDA 进行降维。在降维后的数据上训练一个分类器（如 SVM），并评估分类性能。

示例代码：

```
8 # 生成数据集
9 X, y = make_classification(
10     n_samples=1000,
11     n_features=50,
12     n_informative=4,
13     n_redundant=20,
14     n_repeated=10,
15     n_classes=3,
16     flip_y=0.2,
17     random_state=42
18 )
19
20 # 划分数据集
21 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
22
23 # LDA降维后分类
24 lda = LDA(n_components=2)
25 X_train_lda = lda.fit_transform(X_train, y_train)
26 X_test_lda = lda.transform(X_test)
27
28 svm_lda = SVC(kernel='linear', C=0.1) # 调整正则化参数
29 svm_lda.fit(X_train_lda, y_train)
30 accuracy_lda = accuracy_score(y_test, svm_lda.predict(X_test_lda))
31
32 # 不降维直接分类
33 svm_orig = SVC(kernel='linear', C=0.1)
34 svm_orig.fit(X_train, y_train)
35 accuracy_orig = accuracy_score(y_test, svm_orig.predict(X_test))
36
37 print(f'Accuracy after LDA: {accuracy_lda:.2f}')
38 print(f'Accuracy without reduction: {accuracy_orig:.2f}')
```

输出结果如下所示：

Accuracy after LDA: 0.56

Accuracy without reduction: 0.54

3. t-SNE 降维及数据可视化

t-SNE 是一种非线性降维技术，通过将高维数据点间的相似性关系映射到低维空间实现可视化。其核心是：在高维空间用高斯分布衡量邻近点相似度，在低维空间用 t 分布（长尾特性）重新建模这些关系，并通过优化使两者分布尽可能匹配。这种设计使得相近的数据点在低维中紧密聚集，而原本远离的点自动分离，尤其擅长保留局部结构（如簇状分布），因此广泛用于图像、文本等复杂数据的二维/三维可视化。

设计一个实验以 MNIST 手写数字数据集或其他数据集为载体，通过 t-SNE 方法将 784 维图像数据压缩至二维平面，验证其能否在保留同类数据局部聚集特性的同时，实现不同数字类别的自然分离，从而可以直观的观察高维非线性数据在低维空间中的可解释性。